Массив сообщений через HTTP цикл в LEADTEX

Практическая схема отправки каждого элемента JSON массива отдельным сообщением Telegram



Главный вывод. Количество блоков не зависит от длины массива. Один блок получает весь JSON, второй отправляет текущий элемент и вызывает себя через goToBlock().

Сценарий 2352423 Блок получения 49905282 Блок отправки 49905288

1 Общая логика

Массив остаётся неизменным на протяжении всей серии. Переменная demo_index показывает, какой элемент нужно подготовить следующим. После успешной отправки индекс увеличивается на единицу.

Шаг	Действие	Результат
1	GET к внешнему API	Получен объект с items[]
2	Сохранение массива	demo_items содержит JSON строку
3	Начальный индекс	demo_index = 0
4	POST в Telegram	Отправлен items[index]
5	Увеличение индекса	demo_index = demo_index + 1
6	Проверка границы	Повтор или завершение

Условие цикла

```
if (index < items.length) {
  // Есть следующий элемент.
  goToBlock(49905288);
} else {
  // Все элементы отправлены.
  setContactVariable("demo_status", "complete");
}
```

Для массива из трёх элементов существуют индексы 0, 1 и 2. Когда индекс становится равен 3, следующего элемента уже нет и цикл завершается.

Отправлен элемент	Новый индекс	Длина	Продолжить
items[0]	1	3	Да
items[1]	2	3	Да
items[2]	3	3	Нет

2 Ответ внешнего API

Сервер возвращает один объект. Поле items содержит записи, которые нужно превратить в отдельные сообщения.

```
{
  "success": true,
  "count": 7,
  "items": [
    {
      "id": "demo-1",
      "text": "Тест API: сообщение 1 из 7.",
      "button_text": "Открыть исходный JSON",
      "url": "https://pay.sushi-house-39.ru/leadteh-demo/messages-7.json"
    }
  ]
}
```

Поле	Тип	Назначение
success	boolean	Признак успешного ответа нашего API
count	number	Количество элементов для диагностики
items	array	Массив будущих сообщений
items[].text	string	Текст отдельного сообщения
button_text	string	Подпись inline кнопки
url	string	Адрес, открываемый кнопкой

Тестовые размеры массива

- https://pay.sushi-house-39.ru/leadteh-demo/messages-0.json
- https://pay.sushi-house-39.ru/leadteh-demo/messages-7.json
- https://pay.sushi-house-39.ru/leadteh-demo/messages-10.json
- https://pay.sushi-house-39.ru/leadteh-demo/messages-25.json

3 Переменные контакта

goToBlock() переключает блок, но не передаёт аргументы. Поэтому массив, индекс и подготовленное тело запроса хранятся в переменных текущего контакта.

Переменная	Пример	Назначение
demo_items	JSON строка	Полный массив items
demo_total	7	Исходное количество элементов
demo_index	0...7	Индекс следующего элемента
demo_text_json	"Текст"	Экранированный текст для Telegram
demo_markup_json	{inline_keyboard...}	Готовая клавиатура Telegram
demo_status	sending complete	Текущее состояние серии
demo_http_status	200 или 429	HTTP код последнего сбоя
demo_retry_after	3	Пауза Telegram при ограничении скорости
demo_last_message_id	12345	ID последнего сообщения Telegram

Почему используется JSON stringify

LEADTEX хранит контактную переменную как текст. JSON.stringify() превращает массив или объект в строку и одновременно корректно экранирует кавычки и переносы строк. Перед чтением массива второй блок выполняет JSON.parse().

```
setContactVariable("demo_items", JSON.stringify(data.items));

var items = JSON.parse(getContactVariable("demo_items"));
```

4 Блок 01 Получить массив

Метод GET. URL теста: https://pay.sushi-house-39.ru/leadteh-demo/messages-7.json. Таймаут: 5 секунд.

```
// Отключаем автоматический переход «Продолжить».
disableContinue();

// response.data содержит разобранный JSON ответа.
var data = response.data;

// Проверяем HTTP код и контракт нашего API.
if (Number(response.status) !== 200 || !data ||
    data.success !== true || !data.items) {
  setContactVariable("demo_status", "fetch_error");
  setContactVariable("demo_http_status", response.status);
  sendMessage("Не удалось получить массив. HTTP: " + response.status);
} else if (data.items.length === 0) {
  // Пустой массив обрабатываем как отдельное состояние.
  setContactVariable("demo_status", "empty");
  sendMessage("Сервер вернул пустой массив.");
} else {
  // Сохраняем массив и начинаем с элемента 0.
  var item = data.items[0];
  setContactVariable("demo_items", JSON.stringify(data.items));
  setContactVariable("demo_total", data.items.length);
  setContactVariable("demo_index", 0);
  setContactVariable("demo_status", "sending");

  // Подготавливаем JSON текст и кнопку первого сообщения.
  setContactVariable("demo_text_json", JSON.stringify(item.text));
  setContactVariable("demo_markup_json", JSON.stringify({
    inline_keyboard: [[{
      text: item.button_text,
      url: item.url
    }]]
  }));

  // Передаём управление блоку отправки.
  goToBlock(49905288);
}
```

Что происходит после успешного ответа

- Массив сохраняется целиком и не изменяется.
- Индекс устанавливается в 0, то есть выбирается первый элемент.
- Текст и клавиатура превращаются в корректные JSON фрагменты.
- Блок отправки запускается через goToBlock(49905288).

5 JSON тело запроса Telegram

```
{
  "chat_id": "{{telegram_id}}",
  "text": {{$demo_text_json}},
  "reply_markup": {{$demo_markup_json}},
  "disable_notification": true
}
```

Поле	Источник	Почему так
chat_id	{{telegram_id}}	Стабильный Telegram ID текущего контакта
text	{{$demo_text_json}}	Переменная уже содержит JSON строку с кавычками
reply_markup	{{$demo_markup_json}}	Переменная уже содержит JSON объект
disable_notification	true	Тестовая серия приходит без звука

Вокруг demo_text_json и demo_markup_json нельзя добавлять ещё одну пару кавычек. Иначе Telegram получит строку вместо текста или объекта клавиатуры.

Успешный ответ Telegram

```
{
  "ok": true,
  "result": {
    "message_id": 12345
  }
}
```

Проверяются оба признака: HTTP 200 и data.ok === true. После этого можно увеличить индекс и перейти к следующему элементу.

6 Блок 02 Отправить и повторить

```
// Повтор контролируется кодом, а не связью «Продолжить».
disableContinue();
var data = response.data;

// Проверяем транспортный и прикладной результат Telegram.
if (Number(response.status) !== 200 || !data || data.ok !== true) {
  setContactVariable("demo_status", "send_error");
  setContactVariable("demo_http_status", response.status);
  setContactVariable(
    "demo_retry_after",
    data && data.parameters ? data.parameters.retry_after || 0 : 0
  );
  sendMessage("Тест остановлен на индексе " +
    getContactVariable("demo_index"));
} else {
  // Восстанавливаем массив и отмечаем успешную отправку.
  var items = JSON.parse(getContactVariable("demo_items"));
  var index = Number(getContactVariable("demo_index")) + 1;
  setContactVariable("demo_index", index);
  setContactVariable("demo_last_message_id", data.result.message_id);

  // Продолжаем, если есть следующий элемент и не достигнут предел.
  if (index < items.length && index < 25) {
    var item = items[index];
    setContactVariable("demo_text_json", JSON.stringify(item.text));
    setContactVariable("demo_markup_json", JSON.stringify({
      inline_keyboard: [[{
        text: item.button_text,
        url: item.url
      }]]
    }));
    goToBlock(49905288);
  } else {
    setContactVariable("demo_status", "complete");
  }
}
```

Защитный предел

Условие index < 25 ограничивает одну серию максимум двадцатью пятью сообщениями. Это страховка от ошибочно большого массива или бесконечного повторного запуска. Для другого рабочего предела число можно вынести в отдельную переменную.

7 Ограничение 20 блоков

LEADTEX ограничивает цепочку из блоков, которые выполняются подряд без участия пользователя. В этой реализации через визуальную связь «Продолжить» проходят только старт, вступление и получение массива.

Часть	Механизм перехода	Количество блоков
Запуск	Связь Продолжить	3
Первая отправка	goToBlock(49905288)	1 HTTP блок
Следующие отправки	Самопереход goToBlock	Тот же HTTP блок

Живой тест на семи элементах завершился успешно. Тестовый ответ на 25 элементов подготовлен, но не запускался, поэтому поведение повторного вызова после двадцатой отправки пока не подтверждено фактическим прогоном.

8 Как проверить

1. Открыть https://app.leadteh.ru/w/giT34.
2. Запустить Telegram бота, если он ещё не запускался этим пользователем.
3. Убедиться, что пришли семь отдельных сообщений и семь inline кнопок.
4. Проверить demo_total = 7, demo_index = 7 и demo_status = complete.
5. При ошибке проверить demo_http_status и demo_retry_after.

Результат живого теста

10 сентября 2026 года сценарий отправил все семь элементов массива отдельными сообщениями Telegram. Каждое сообщение содержало кнопку, сформированную из полей button_text и url соответствующего элемента.